DATABASE USAGE

SAE S2.04

Training/ Educational Establishment

IUT of Saint-Dié-Des-Vosges - First year

BUT Informatique

Written by

Antoine Barthélémy, Lucas Cesar

Tutor

Fahed Abdallah, Aline Caspary,
Patrick Adelbrecht

UNIVERSITÉ DE LORRAINE | IUT Saint-Dié-des-Vosges

Introduction

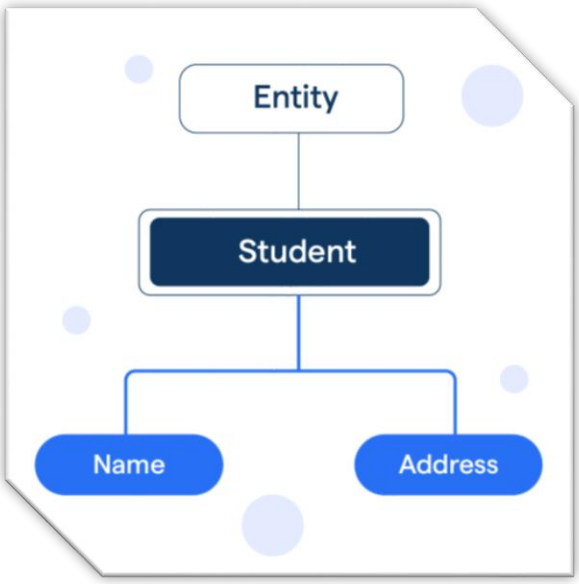
In the second semester of our BUT Informatique at the IUT of Saint-Dié-des-Vosges, we had to produce a report for the SAE “Database Usage”, this SAE is part of the Teaching Unit 4. The main objectives was to create a database through several steps, each involving different tools. We chose to work on a database for a supermarket because we found it both interesting and achievable. The objectives of this SAE are to improve our ability to use mathematical tools from the field of Statistics and to develop our skills in data manipulation. You will find detailed explanations throughout the document explaining the way in which we’ve operrated and reasoned. Regarding the subject’s choice, the supermarket database was a relevant choice because it involves a variety of entities and numerical data, which are essential for the modeling and analysis tasks required. This SAE aimed to strengthen our technical skills in databse design, SQL programming, and statistical analysis, as well as to develop our ability to handle data effectively using different tools.

Analysis and research

Introduction

The goal of this step is to identify the entities, attributes, and associations needed for a supermarket database. We conducted online research to understand the typical database structures used in supermarkets, while explaining our choices and citing our sources. It is important to note that the database is not designed for a customer-facing website or application, but rather for the supermarket’s comprehensive internal system. This explains the inclusion of entites such as Employee.

A BRIEF REMINDER OF THE NATURE AND ROLE OF AN ENTITY:



An entity is a "thing" or "object" from the real world. It is described by a set of attributes that characterize it. An entity is essentially a digital or numerical representation of a real-world object in the database. For example, a student, an employee are all entities.

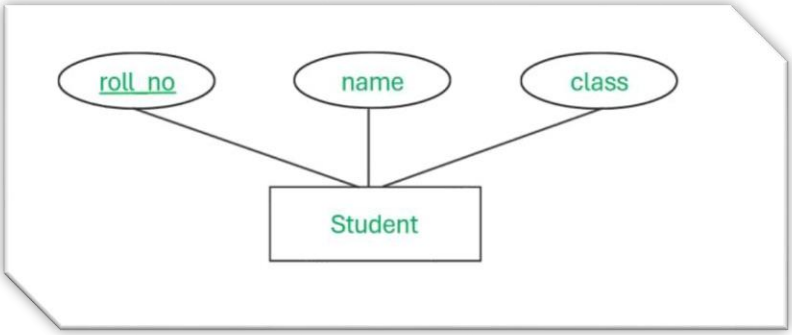
Identification of Entities

- **Customer** -> A customer is represented by attributes such as name, phone number, and so on. Customers will play a central role in our database.
- **Product** -> This entity contains information about products available to customers, such as the product name, expiration date, and available quantity.
- **Bill** -> A bill is generated once a customer has selected products. It serves to link the customer to the products purchased
- **Supplier** -> The supplier entity is essential for managing stock and purchases. A supplier usually has attributes like a supplier id, a phone number etc.
- **Employee** -> The employee entity is necessary for tracking staff roles, responsibilites, and actions within the buisness operations.
- **Department** -> Ideal for performing targeted operations such as the possibility to add new Employe, to check which employee are assigned and so on.
- **Sales** -> Sales entity will compile data about sales that occur in the supermaket. Those information are crucial for the business managers
- **Basket** -> A basket entity links a customer to multiple products before the bill is generated, it is useful to represent temporary selections before the payment happens
- **Promotion** -> A promotion entity could be useful to store information about price reductions for products
- **ProductRating** -> In order to allow multiple customers to rate the same product and to provide flexibility for future extensions, we decided to create a separate ProductRating.
- **Stock** -> Stock ensures product availability for sale, optimizes supply management, and helps prevent stockouts or overstocking.
- **PurchaseOrderSupplier** -> The purchaseOrderSupplier records the products bought from suppliers, including the quantity ordered, the purchase date, and the price paid.
- **BillLine** -> In order to ensure flexibility and maintain a normalized database structure, we chose to create a separate BillLine entity instead os storing product information directly in the Bill entity. By using BillLine, we can accurately model a many-oto-many relationship between Bill and Product, and we make it easier to calculate totals, generate invoices, and analyze saled data more effectively.
- **BasketLine** -> Globally the same reason than the BillLine entity.
- **PromotionProduct** -> In a retail or e-commerce database, a product can be associated with multiple promotions and a promotion can apply to multiple products. This represents a many-to-many relationship, which cannot be directly represented in a relational databse without creating an intermediate table.
- **SalesLine** -> We have decided to introduce this join table since the relation between a product and a sale entity is n-n.

Identification of Attributes and Associations

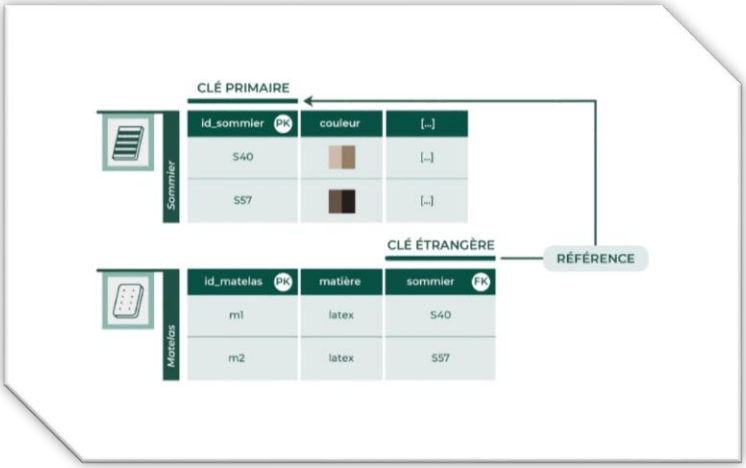
A BRIEF REMINDER OF THE NATURE AND ROLE OF AN ATTRIBUTE:

Attributes are the properties or characteristics of an entity. They provide insightful details about the entity, like specific values that describe or identify it. If we retake our previous example with the student. Students will have multiple attributes such as roll number, name, and class.



A BRIEF REMINDER OF THE NATURE AND ROLE OF AN ASSOCIATION:

An association is a business component that defines a relationship between two entity objects based on common attributes. The relationship can be one-to-one, one-to-many, or many-to-many. The association allows entity objects to access the data of other entity objects through a persistent reference.



Légende:

Italic = PrimaryKey
Bold = ForeignKey
Highlight = Join table

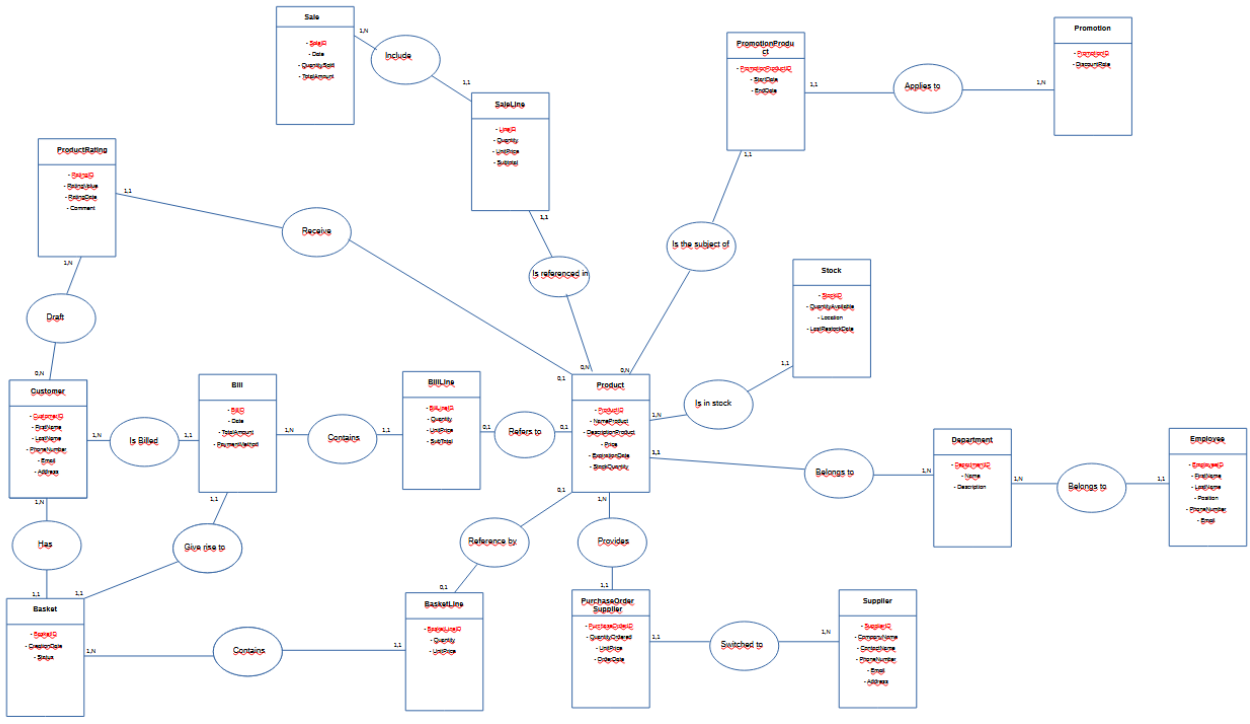
Entities	Attributes	Associations
Customer	*CustomerID* FirstName	- A single customer may not draft any

	LastName PhoneNumber Email Address	product ratings, but they can also draft several - A single customer has at least one basket, even if it is empty, and may have multiple baskets. - A single customer has at least one bill (created by the system after a payment), and may have multiple bills
Product	*ProductID* NameProduct DescriptionProduct Price ExpirationDate StockQuantity	- A single product may not have any rating or a single one - A single product may not appear in any sale, but it can also appear in multiple sale line if it has been sold several times - A single product may not have a promotion but it can also have multiple promotions - A single product is mandatory in stock but it may be multiple times - A single product must be associated with exactly one stock, and it may appear in several stock records over time if restocked or distributed across locations. - A single product must belong to one, and only one, department. Each department can be responsible for one or more products. - A single product can appear in multiple purchase orders, each time ordered from a supplier.

Bill	*BillID* Date TotalAmount PaymentMethod	- Every bill must be associated with one, and only one, customer who created it. - A single bill must contain at least one bill line, and may contain multiple basket lines.
Supplier	*SupplierID* CompanyName ContactName PhoneNumber Email Address	- A supplier can provide multiple products through distinct purchase orders.
Employee	*EmployeeID* FirstName LastName Position PhoneNumber Email	- A single employee must belong to one, and only one, department.
Department	*DepartmentID* Name Description	- A department may include multiple products, but each product must be associated with exactly one department.
Sale	*SaleID* Date QuantitySold TotalAmount	- A single sale must contain at least one sale line, and may contain multiple sale lines, depending on how many products were purchased during that transaction
Basket	*BasketID* CreationDate Status	- Every basket must be associated with one, and only one, customer who created it. - A single basket must contain at least one basket line, and may contain multiple basket lines.
Promotion	*PromotionID* DiscountRate	- A promotion must applies at least on

		one product but can also be applied to several products
ProductRating	*RatingID* RatingValue RatingDate Comment	- Every product rating must be associated with one, and only one, customer who created it - Every product rating must be associated with one, and only one, product
Stock	*StockID* QuantityAvailable Location LastRestockDate	- Each stock record refers to one, and only one product.
PurchaseOrderSupplier	*PurchaseOrderID* QuantityOrdered UnitPrice OrderDate	- Each purchase order line refers to one, and only one, product and one, and only one, supplier.
BillLine	*BillLineID* Quantity UnitPrice SubTotal **BillID** **ProductID**	- Each bill line is associated with exactly one bill, meaning that it cannot exist independently and always belongs to a specific bill.
BasketLine	*BasketLine* Quantity UnitPrice **ProductID** **BasketID**	- Each basket line is associated with exactly one basket, meaning that it cannot exist independently and always belongs to a specific basket.
PromotionProduct	*PromotionProduct* StartDate EndDate **ProductID** **PromotionID**	- A promotion product is associtaed with exactly one product, meaning it cannot exist independently
SaleLine	*LineID* Quantity UnitPrice Subtotal	- Every sale line must be associated with one, and only one, product, and must beloing to one, and only one, sale

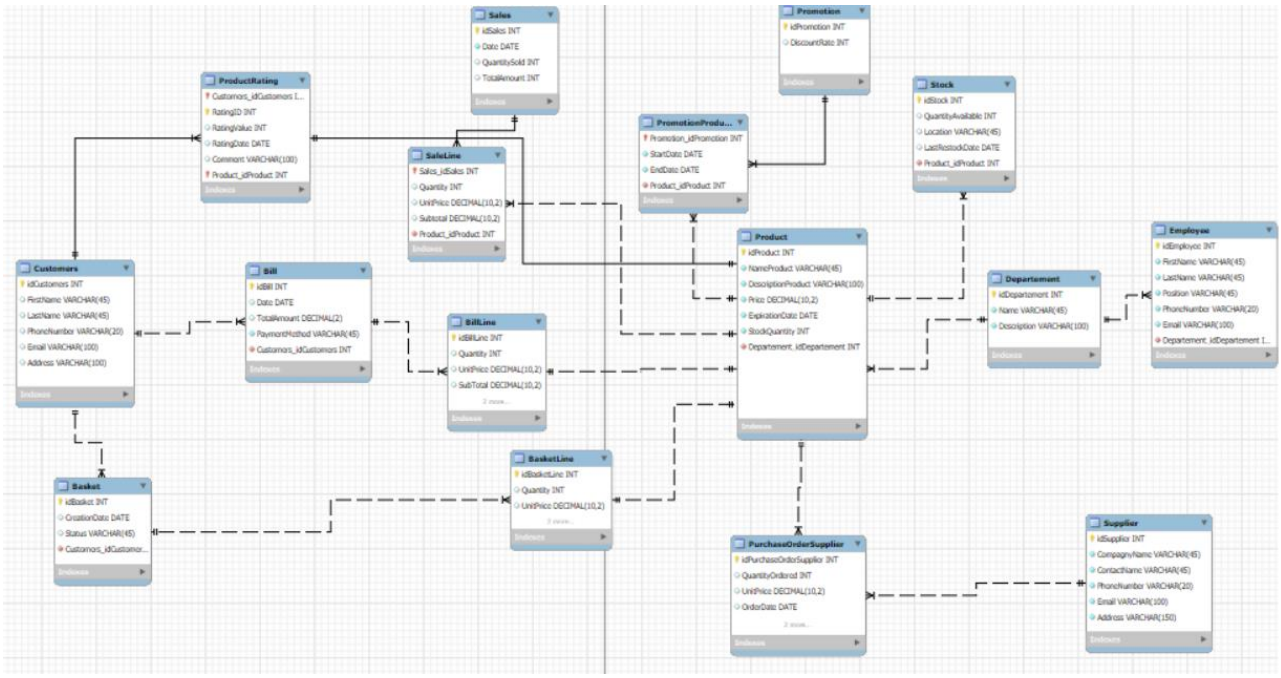
Customer N-------N **ProductRating**
Customer N-------1 **Basket**
Customer N-------1 **Bill**
ProductRating 1-------1 **Product**
Bill N-------1 **BillLine**
Basket N-------1 **BasketLine**
BasketLine 1-------1 **Product**
BillLine 1-------1 **Product**
Product N-------1 **SaleLine**
SaleLine 1-------N **Sale**
Product N-------1 **PurchaseOrderSupplier**
PurchaseOrderSupplier 1-------N **Supplier**
Product N-------1 **PromotionProduct**
PromotionProduct 1-------N **Promotion**
Product N-------1 **Stock**
BillLine 1-------N **Department**
Department N-------1 **Employee**



To better view the E/A model, please review the LibreOffice file in the.zip file.

The relational model was created in a first time with Workbench, as it allows for the automatic inclusion of foreign keys. It is enough to link the different tables with the correct rules of passage.

The Workbench template can be found below, to have a better overview, just open the Workbench file that is located in the.zip folder.



RELATIONAL MODEL (CLASSICAL NOTATION)

Customers (*idCustomers*, FirstName, LastName, PhoneNumber, Email, Address)

Basket (*idBasket*, CreationDate, Status, **idCustomers**)

BasketLine (*idBasketLine*, Quantity, UnitPrice, **idBasket**, **idProduct**)

Bill (*idBill*, Date, TotalAmount, PaymentMethod, **idCustomers**)

BillLine (*idBillLine*, Quantity, UnitPrice, SubTotal, **idBill**, **idProduct**)

Product (*idProduct*, NameProduct, DescriptionProduct, Price, ExpirationDate, StockQuantity, **idDepartement**)

ProductRating (*idCustomers*, *RatingID*, *idProduct*, RatingValue, RatingDate, Comment)

Promotion (*idPromotion*, DiscountRate)

PromotionProduct (*idPromotion*, StartDate, EndDate, **idProduct**)

PurchaseOrderSupplier (*idPurchaseOrderSupplier*, QuantityOrdered, UnitPrice, OrderDate, **idSupplier**, **idProduct**)

Sales (*idSales*, Date, QuantitySold, TotalAmount)

SaleLine (*idSales*, Quantity, UnitPrice, Subtotal, **idProduct**)

Stock (*idStock*, QuantityAvailable, Location, LastRestockDate, **idProduct**)

Departement (*idDepartement*, Name, Description)

Employee (*idEmployee*, FirstName, LastName, Position, PhoneNumber, Email, **idDepartement**)

Supplier (*idSupplier*, CompagnyName, ContactName, PhoneNumber, Email, Address)

Legend:

- The bold attributes are foreign keys.
- Italic attributes are primary keys.

IN-DEPTH EXPLANATIONS REGARDING THE CARDINALITIES 'CHOICES.

The Customer entity is directly linked to the ProductRating entity, as we have decided to allow customers to draft ratings without requiring any payment. This reflects common real-world scenarios, such as on platforms like Amazon, where users can leave reviews without having made a purchase.

Even though it might seem surprising at first to see a direct connection between the Customer and Bill entities — since the customer doesn't actually generate the bill — this is entirely correct. In reality, it is the system that generates the bill once the payment is made.

We thought it would make sense to link the ProductRating entity with the Product entity using these cardinalities. Indeed, on the Product side of the relationship, customers will find summarized information derived from the ProductRating entity, which justifies the (0,1) cardinality on the Product side.

Our sales entity stores information intented for managerial use. It allows managers to review what has been sold over a given period, such as a year. We chose to include this entity because our project involves applying statistical tool, and we believed it would be valuable to support data analysus and reporting. In order to do so, it is necessary to create a join table.

Database feed:

At the level of feeding the database we have about insert 50 rows of data per tables. We will show you some examples of insertion line in order to avoid overloading the Word document for nothing because it is always the same command according to the table that we want to fill.

Examples of INSERT include:

```
INSERT INTO Basket (CreationDate, Status, Customers_idCustomers) VALUES ('2024-01-03', 'payé', 1);
INSERT INTO Basket (CreationDate, Status, Customers_idCustomers) VALUES ('2024-01-05', 'en attente', 2);

INSERT INTO Employee (FirstName, LastName, Position, PhoneNumber, Email, Departement_idDepartement) VALUES ('Laura', 'Lemoine', 'Caissière', '0600011122', 'laura.lemoine@supermarche.fr', 1);

INSERT INTO PromotionProduct (Promotion_idPromotion, StartDate, EndDate, Product_idProduct) VALUES
(1, '2025-03-01', '2025-03-15', 1),
(2, '2025-03-10', '2025-03-25', 2);
```

ETC…

You can find the complete script of the database in the folder . zip, there will be all the tables with all the data inside.

Triggers

A trigger is a stored procedure in a database that automatically invokes whenever a special event in the database occurs. By using SQL triggers, developers can automate tasks, ensure data consistency, and keep accurate records of database activities. For instance, a trigger can stop someone from adding 15 bananas to their basket if only 5 in stock and so on.

We will be focusing on high-value, essential triggers based on three criteria:

1. Business Criticality (data integrity, preventing loss, core processes)
2. Impact on User experience (smooth checkout, valid promotions, etc.)
3. Technical Simplicity vs Value (easy to implement but high benefit)

Now that we have defined our criteria, we can identify which triggers are relevant. We have made a list of 5 triggers for you. Here are the one we've talked about :

- Update stock after a sale -> Reduces product quantity in stock, crucial for availability and restocking logic.
- Update bill total after adding a billine -> Critical for accurate billing and customer trust
- Apply active promotion to saleline -> Automatically adjusts price if a promotion exists
- Check stock availability before adding to basketline -> Avoids customers adding more items than available
- Prevent deletion of products in use -> Avoids to lose information on other table where the product is referenced.

There are three types of triggers DDL Triggers, DML Triggers and Logon Triggers. As you may know we will work with the DML triggers since their role is more tailored to our expectations, especially in a supermarket database. DML triggers fire when we manipulate data with commands like insert, update, or delete which is exactly what we want. The syntax to create a trigger is the following :

```
CREATE TRIGGER trigger_name
{BEFORE | AFTER} {INSERT | UPDATE| DELETE }
ON table_name FOR EACH ROW
trigger_body;
```

These triggers do not cover every possible case, but they are a solid starting point and add an extra layer of protection against potential misuse or attacks.

```
DELIMITER $$

CREATE TRIGGER updateStockAfterSale
AFTER INSERT ON SaleLine
FOR EACH ROW
BEGIN
  UPDATE Stock
  SET QuantityAvailable = QuantityAvailable - NEW.Quantity
  WHERE Product_idProduct = NEW.Product_idProduct;
END $$

DELIMITER ;
```

To test the trigger you need: identify a product and its stock then look at the quantity available, insert a sales line and then re-check the stock.

```
DELIMITER $$
```

```
CREATE TRIGGER updateBillTotalAfterAddingBillLine
AFTER INSERT ON BillLine
FOR EACH ROW
BEGIN
   UPDATE Bill
   SET TotalAmount = TotalAmount + NEW.SubTotal
   WHERE idBill = NEW.Bill_idBill;
END $$

DELIMITER;
```

In order to test this trigger you must: display the total of an invoice (TotalAmount), add a new line in BillLine for this invoice.
Then check that the total of the bill to increase.

```
DELIMITER $$

CREATE TRIGGER applyPromotionAutomatically
BEFORE INSERT ON SaleLine
FOR EACH ROW
BEGIN
   DECLARE productPrice DECIMAL(10,2);
   DECLARE discountRate DECIMAL(5,2) DEFAULT 0;

   SELECT Price INTO productPrice
   FROM Product
   WHERE idProduct = NEW.Product_idProduct;

   SELECT p.DiscountRate INTO discountRate
   FROM Promotion p
   JOIN PromotionProduct pp ON pp.Promotion_idPromotion = p.idPromotion
   WHERE pp.Product_idProduct = NEW.Product_idProduct
    AND CURRENT_DATE BETWEEN pp.StartDate AND pp.EndDate
   LIMIT 1;

   SET NEW.UnitPrice = productPrice * (1 - discountRate);
   SET NEW.Subtotal = NEW.UnitPrice * NEW.Quantity;
END $$

DELIMITER ;
```

To test this trigger, just insert a line in my SaleLine table without defining UnitPrice and Subtotal.
Then check the result.

```
DELIMITER $$

CREATE TRIGGER checkStockBeforeBasketInsert
```

```
BEFORE INSERT ON BasketLine
FOR EACH ROW
BEGIN
  DECLARE availableQuantity INT;

  SELECT SUM(QuantityAvailable) INTO availableQuantity
  FROM Stock
  WHERE Product_idProduct = NEW.Product_idProduct;

  IF NEW.Quantity > IFNULL(availableQuantity, 0) THEN
    SIGNAL SQLSTATE '45000'
    SET MESSAGE_TEXT = 'Not enough stock available for this product.';
  END IF;
END $$

DELIMITER ;
```

To test this trigger, it is enough to check the stock of a product, make a normal insertion in the basketLine table (sufficient stock) and make another insertion with an insufficient stock and we get the error.

```
DELIMITER //

CREATE TRIGGER preventProductDeletion
BEFORE DELETE ON Product
FOR EACH ROW
BEGIN
  DECLARE countBillLines INT DEFAULT 0;
  DECLARE countSaleLines INT DEFAULT 0;
  DECLARE countPromotions INT DEFAULT 0;

  SELECT COUNT(*) INTO countBillLines
  FROM BillLine
  WHERE Product_idProduct = OLD.idProduct;

  IF countBillLines > 0 THEN
    SIGNAL SQLSTATE '45000'
    SET MESSAGE_TEXT = 'Cannot delete product: it is used in a bill.';
  END IF;

  SELECT COUNT(*) INTO countSaleLines
  FROM SaleLine
  WHERE Product_idProduct = OLD.idProduct;

  IF countSaleLines > 0 THEN
    SIGNAL SQLSTATE '45000'
    SET MESSAGE_TEXT = 'Cannot delete product: it is used in a sale.';
  END IF;
```

```
    SELECT COUNT(*) INTO countPromotions
    FROM PromotionProduct
    WHERE Product_idProduct = OLD.idProduct;

    IF countPromotions > 0 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Cannot delete product: it is linked to a promotion.';
    END IF;
END;
//

DELIMITER ;
```

To test this last trigger, we must try to delete a product used and we get the defined error.

Exploratory data analysis

Introduction

Exploratory data analysis is an important step for any company that wants to get clear results on trends, anomalies and significant phenomena that affect it. Within the framework of this database dedicated to a supermarket, such a mathematical analysis can allow, for example, to better understand purchasing behaviors, distribution of products sold, and many other aspects.

We will explore, through a series of relevant statistics accompanied by explicit graphs, the phenomena that govern our database. The progression of sections in this part of the SEA will follow a clear and gradual logic, with particular emphasis on an interpretation of results that is accessible even to a non-expert audience.
Les objectifs de cette partie sont donc :

- to intelligently explore data
- to choose the right statistical and graphical indicators
- to justify our choices based on our base
- and clearly communicate the results to a non-expert reader

Methodology

We chose to use the Python language, relying mainly on the pandas, matplotlib and seaborn libraries. Python was chosen for its syntactic clarity, its popularity in the fields of data science and data analysis, as well as the wealth of resources available. On a personal level, I have already used Python several times, which allows me to gain efficiency and autonomy for this analysis.

Given the time constraints and the desire to propose a clear and understandable analysis, we have selected indicators with high added value for a non-expert audience, while remaining close to the business concerns of a supermarket. The focus is on average expenses, product performance, sales performance and inventory management, which are all essential elements of decision-making in a business management context.

The indicators common to several analyses are:
- Average
- Median
- Standard Deviation
- Minimum/Maximum
- Total amount
- Frequency / Proportion
- Mode

These indicators will be used in our exploratory analysis. We have listed for you in a table the summary of priority analyses. But before I describe it, here are the three criteria for their selection:

1. **Business relevance:** does this calculation help a decision maker to make a decision or understand a situation?
2. **Readability:** is the result easily understood by a non-expert?
3. **Speed of implementation:** is the calculation simple to code or explain?

Important note: The mathematical indicators and visualization tools presented below are intended as toolboxes. During the concrete analysis (part 3), we will select from these tools those that will be actually used, depending on the nature of the data and the relevance of the indicator for each case. The use of several tools is not mandatory, but can be complementary if it brings a better understanding.

The associated table is:

Analysis	Objective	Table(s)	Main Field(s)	Selected Statistical Indicators	Suggested Visualization Tools	Nature of Variables
Average basket	Understand the average spending per customer	Bill	TotalAmount	Mean, Median, Standard deviation, Min, Max	Histogram, Boxplot	Quantitative Continuous
Critical stock levels	Detect products close to stock depletion	Stock	QuantityAvailable	Minimum, Critical threshold, Ranking	Bar chart, Filtered table	Quantitative Discrete

Sales vs. stock tension	Identify best-selling products and compare with stock	BillLine / Stock	Quantity (sold), QuantityAvailable (stock)	Total sum, Cross-variable comparison	Crossed bar chart, Comparison table	Pair of Quantitative Discrete values

Analysis by key attributes

ANALYSIS: AVERAGE BASKET

1. Selection of indicators
   - Among the indicators proposed in methodology, I use the average and standard deviation.
   - I chose the average over the median because the dataset does not have outliers.
   - The standard deviation is used to measure the extent to which basket amounts are homogeneous or dispersed. It is therefore useful to assess the overall consistency of purchases
   - The minimum and maximum indicators were not selected here because they do not provide useful information in this context: no outliers were identified, and the objective of the analysis is to identify a general trend rather than isolated cases.

2. Calculations
   - Rq: The code to achieve its results is present in Appendix + the calculations given below are built according to the statistical distribution (visualization part)

$$\bar{x} = \sum f_i \cdot x_i$$

The average is given by Python code. It is calculated using the following formula: (0.02 × 7) + (0.04 × 9) + (0.06 × 10) … + (0.02 × 64) = 28,02 €.

$$V(x) = \sum f_i \cdot (x_i - \bar{x})^2$$

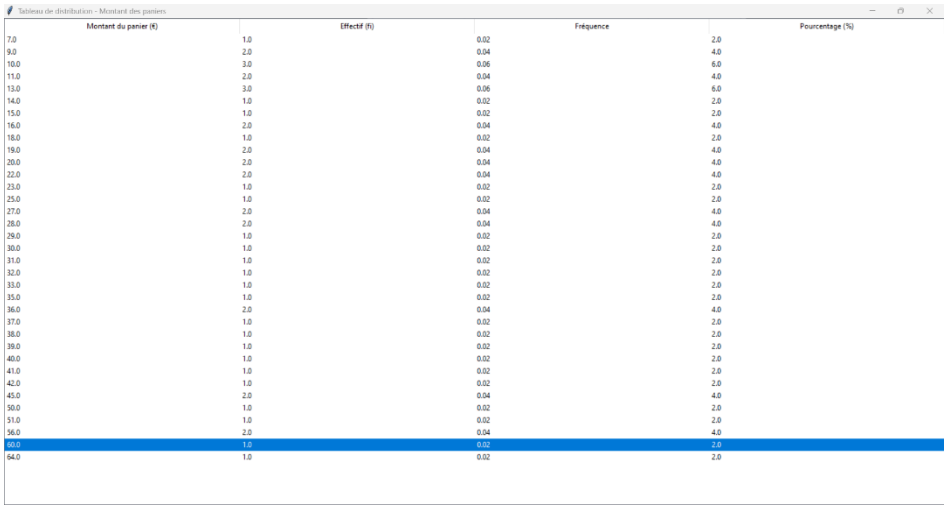
The variance is given by this formula. It is an intermediate step before finding what we are interested in: the standard deviation. The variance measures the dispersion of the squared values (it is the mean of the squares of the deviations to the average).
$0{,}02 \times (7 - 28{,}02)^2 + 0{,}04 \times (9 - 28{,}02)^2 + \ldots + 0{,}02 \times (64 - 28{,}02)^2 = 231{,}64€^2$

$$\sigma = \sqrt{V(x)}$$

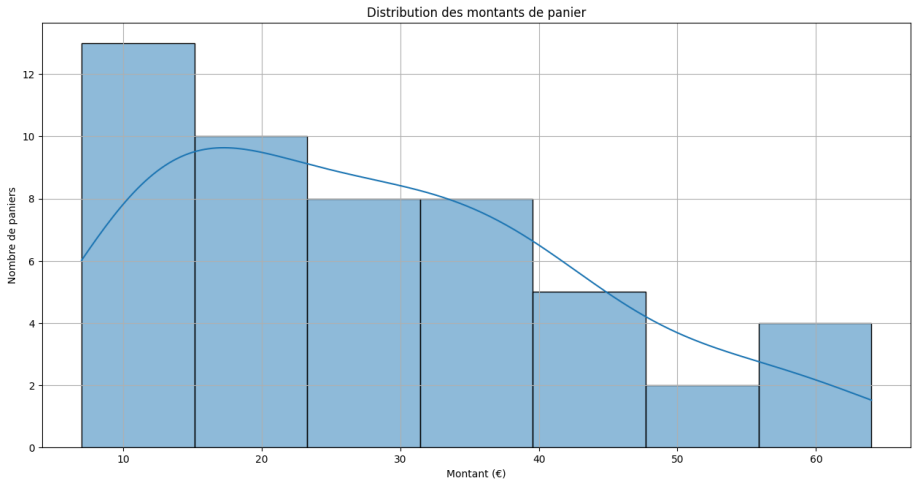
The standard deviation measures the dispersion of values without the power. $\sqrt{231{,}64} = 15{,}22€$

- The average of 28.02 € (a little pulled up by some very expensive baskets) and a high standard deviation (15,22), showing significant variability between clients.

3. Visualization



Montant du panier (€)	Effectif (fi)	Fréquence	Pourcentage (%)
7.0	1.0	0.02	2.0
9.0	2.0	0.04	4.0
10.0	3.0	0.06	6.0
11.0	2.0	0.04	4.0
13.0	3.0	0.06	6.0
14.0	1.0	0.02	2.0
15.0	1.0	0.02	2.0
16.0	2.0	0.04	4.0
18.0	1.0	0.02	2.0
19.0	2.0	0.04	4.0
20.0	2.0	0.04	4.0
22.0	2.0	0.04	4.0
23.0	1.0	0.02	2.0
25.0	1.0	0.02	2.0
27.0	2.0	0.04	4.0
28.0	2.0	0.04	4.0
29.0	1.0	0.02	2.0
30.0	1.0	0.02	2.0
31.0	1.0	0.02	2.0
32.0	1.0	0.02	2.0
33.0	1.0	0.02	2.0
35.0	1.0	0.02	2.0
36.0	2.0	0.04	4.0
37.0	1.0	0.02	2.0
38.0	1.0	0.02	2.0
39.0	1.0	0.02	2.0
40.0	1.0	0.02	2.0
41.0	1.0	0.02	2.0
42.0	1.0	0.02	2.0
45.0	2.0	0.04	4.0
50.0	1.0	0.02	2.0
51.0	1.0	0.02	2.0
56.0	2.0	0.04	4.0
60.0	1.0	0.02	2.0
64.0	1.0	0.02	2.0



4. Interpretation

The chart below shows a histogram representing the distribution of basket amounts in the database. There is a high concentration of baskets in the 10-25€ range, which corresponds to the most frequent purchases by customers.

The superimposed density curve makes it possible to visualize the overall shape of the distribution, which is asymmetrical towards the right: this means that the majority of customers spend little, while a few make higher baskets, up to 60 €.Cette observation est confirmée par les indicateurs statistiques calculés :

- Average: 28,02 €
- Standard deviation: 15,02 €

The high standard deviation indicates a high variability in basket amounts: customers do not all spend the same.

Using the average standard deviation range, it can be estimated that most of the amounts are between € 13.00 and € 43.04, which confirms a significant dispersion around the average.

ANALYSIS: CRITICAL STOCKS

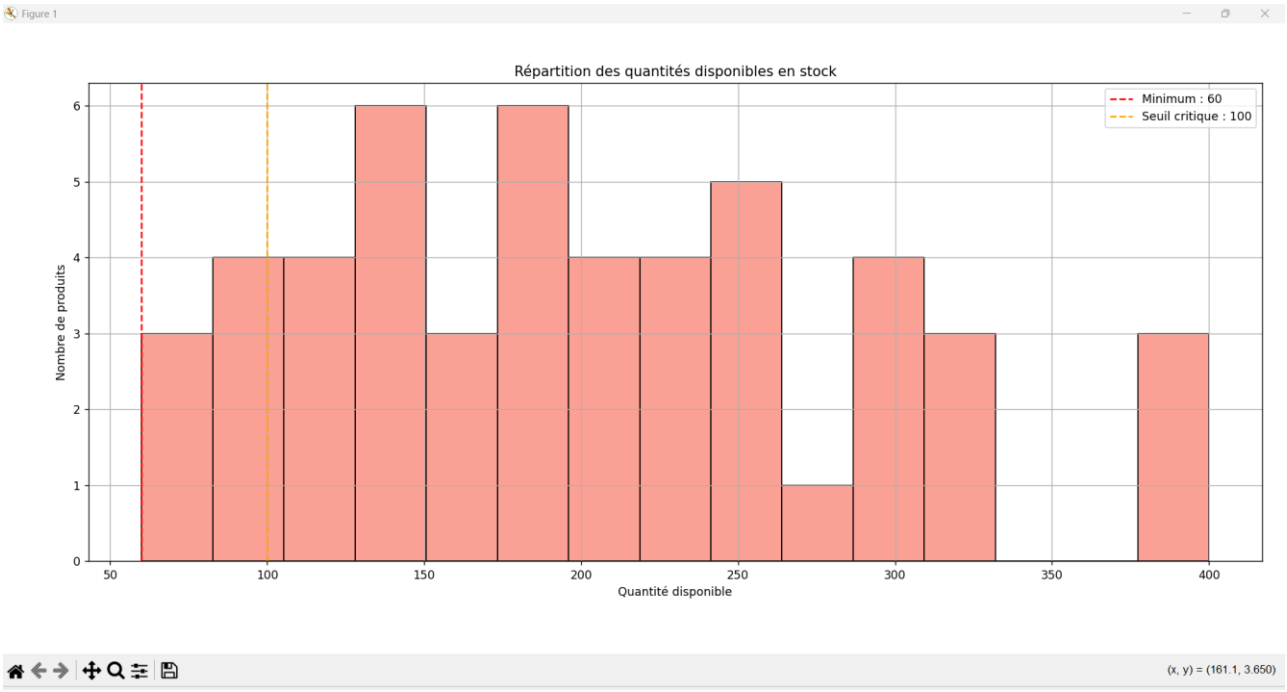
1. Selection of indicators

• Among the indicators proposed in methodology, I use the minimum, critical threshold and ranking
• I chose the minimum because it allows to identify directly the lowest stocks
• The critical threshold to identify urgent situations
• A decreasing ranking of products by stock available, in order to prioritize actions to be undertaken for decision-makers.Calculs

- Rq : The code to achieve its results is present in Appendix + It is not necessary to make a statistical distribution because the values are discreet and few.

$\min(QuantiyAvailable) = 60$

2. Visualization



Top 5 des stocks les plus faibles

idStock	QuantityAvailable	Product_idProduct	Location
8	60	8	Rayon B1
7	70	7	Rayon C2
24	70	24	Entrepôt B1
5	90	5	Rayon D1
20	90	20	Rayon B5

3. Interpretation

The analysis of the distribution of available quantities in stock makes it possible to observe the overall structure of supply levels. The graph shows a moderate concentration around certain values (in particular between 120 and 250 units), but also shows the existence of products in more critical situations.

It is noted that three products are below the critical threshold – this constitutes a warning signal in stock management. This visualization can be useful for a decision-maker, as it helps to anticipate the risks of out of stock.

ANALYSIS: TENSION BETWEEN SALES AND STOCK

1. Selection of indicators

   Our objective is to determine whether there is a correlation between the quantity of a product sold and the quantity available in stock. We will therefore introduce a new formula (out of program): the Pearson linear correlation coefficient between two variables.
   Note: not to be confused with the Pearson asymmetry coefficient, given by the following formula:

$$\text{Asymétrie (skewness)} = \frac{\bar{x} - \text{mode}}{\sigma}$$

   With the correlation one which is used on two quantitative variables:

$$r = \frac{\sum (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum (x_i - \bar{x})^2 \cdot \sum (y_i - \bar{y})^2}}$$

- Naturally, we are led to use a well-known tool: the average, which will be used to complete the formula.

1. Calculations

- Rq: The code to achieve its results is present in Appendix + the calculations given below are built according to the statistical distribution (visualization part)

$$\bar{x} = \sum f_i \cdot x_i$$

Average of Quantity: (0.280*1)+(0.280*2)… + (0.080 *5) = 2.48. On average when a product is purchased, the consumer takes 2.

Average of QuantityAvailable : (0.080 *292)+(0.120*293)… + (0.060 * 298) = 295.16. On average the available quantity of products in stock is 295 products.

$$r = \frac{\sum (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum (x_i - \bar{x})^2 \cdot \sum (y_i - \bar{y})^2}}$$

Pearson's coefficient: (1-2.48)(298-295.16)+(1-2.48)(297-295.16)+….. + (x_n -2.48)(y_n-295.16) / $\sqrt{[(1-2.48)^2 + (1-2.48)^2 + \cdots .] * [(298-295{,}16)^2 + (297-295{,}16)^2 + \cdots}$ = -0.6799

2. Visualization

Statistical distribution on billLine table

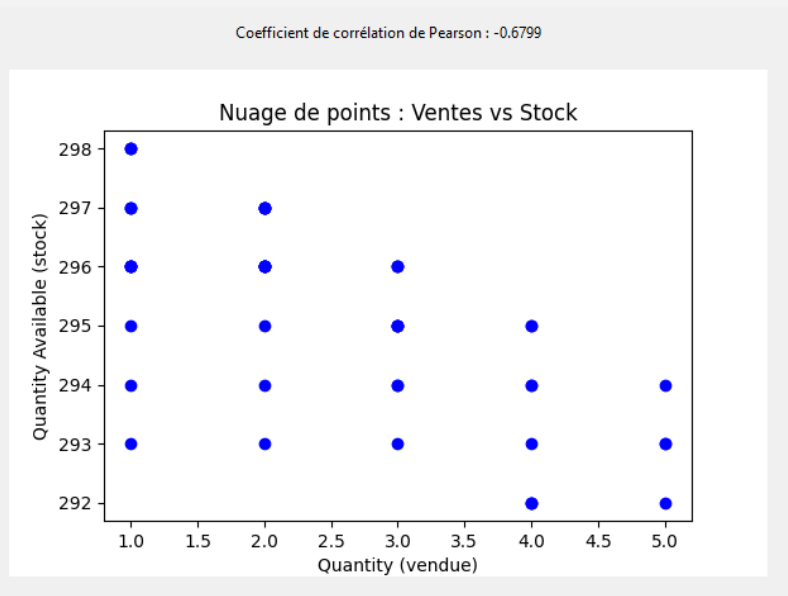
Distribution statistique - Quantity

Xl	Effectif	Fréquence	Pourcentage (%)
1	14	0.280	28.00
2	14	0.280	28.00
3	10	0.200	20.00
4	8	0.160	16.00
5	4	0.080	8.00

Statistical distribution on the stock table

Distribution statistique - QuantityAvailable (Stock)

Xl	Effectif	Fréquence	Pourcentage (%)
292	4	0.080	8.00
293	6	0.120	12.00
294	7	0.140	14.00
295	8	0.160	16.00
296	14	0.280	28.00
297	8	0.160	16.00
298	3	0.060	6.00



3. Interpretation

The more a product is sold, the lower the available stock tends to be. This is confirmed by the Pearson correlation coefficient. This confirms a real tension between sales and stocks, which can lead to a breakdown if supply is not adjusted.

Conclusion

This SAE allowed us to put into practice a wide range of skills acquired during the semester, from database modeling to advanced statistical analysis. Designing a relational database for a supermarket was both a technical and logical challenge, involving the creation of numerous entities and relationships while ensuring data integrity and consistency.

The implementation of DML triggers helped us simulate real-world constraints and improve the reliability of the system. We also fed our database with realistic and diversified data to enhance the relevance of the statistical analyses.

On the analytical side, we were able to extract meaningful insights about customer behavior, inventory management, and sales performance. This step underlined the importance of data in supporting strategic business decisions.

Overall, this project helped us reinforce our mastery of SQL, our capacity to structure complex datasets, and our ability to produce valuable indicators using Python and data visualization tools.

It was a comprehensive and enriching experience that combined technical precision with business-oriented thinking — a valuable preparation for real-world data projects.

Sources:

https://creately.com/diagram/example/hdvcvt4u2/supermarket-database
https://study.com/academy/lesson/what-is-an-entity-in-a-database.html
https://www.geeksforgeeks.org/entity-in-dbms/
https://onecompiler.com/mysql/3zeab2aqh
https://docs.oracle.com/cd/B13224_01/web.904/b10390/bc_awhatisanassoc.htm#:~:text=An%20association%20is%20a%20business,objects%20through%20a%20persistent%20reference.
https://openclassrooms.com/fr/courses/7818671-requetez-une-base-de-donnees-avec-sql/7883636-identifiez-les-types-dassociations-entre-vos-tables
https://www.deepl.com/en/translator
https://chatgpt.com/
https://tadabase.io/blog/using-join-tables#:~:text=A%20join%20table%20is%20a,tables%20to%20one%20data%20table.
https://www.youtube.com/watch?v=0jOUX9G-ZHc&ab_channel=ELOAFORMATION
https://www.youtube.com/watch?v=aw1Q47vSaCc&ab_channel=Grafikart.fr
https://sql.sh/cours/create-trigger
https://www.geeksforgeeks.org/sql-trigger-student-database/
https://www.w3schools.com/sql/func_mysql_if.asp

Annexes :

ANALYSE : PANIER MOYEN

```
import mysql.connector
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import tkinter as tk
from tkinter import ttk

# Connexion à ta base MySQL
connexion = mysql.connector.connect(
    host="localhost",  # ou l'adresse de ton serveur
    user="root",  # ou ton nom d'utilisateur
    password="",  # à adapter
    database="supermarche",  # ex : mydb
)
```

```
curseur = connexion.cursor()
requete = "SELECT * FROM Bill"  # Exemple avec la table Bill
curseur.execute(requete)
colonnes = [desc[0] for desc in curseur.description]
resultats = curseur.fetchall()
df = pd.DataFrame(resultats, columns=colonnes)

df["TotalAmount"] = df["TotalAmount"].astype(float)

# Création du tableau de distribution (effectif, fréquence, pourcentage)
effectifs = df["TotalAmount"].value_counts().sort_index()
frequences = effectifs / effectifs.sum()
pourcentages = frequences * 100

tableau_distribution = pd.DataFrame(
    {
        "Montant du panier (€)": effectifs.index,
        "Effectif (fi)": effectifs.values,
        "Fréquence": frequences.round(4),
        "Pourcentage (%)": pourcentages.round(2),
    }
)
print("Somme des fréquences :", tableau_distribution["Fréquence"].sum())

print("\nTableau de distribution :")
print(tableau_distribution)

# Création de la fenêtre principale
fenetre = tk.Tk()
fenetre.title("Tableau de distribution - Montant des paniers")

# Création du tableau (Treeview)
tableau = ttk.Treeview(fenetre)
tableau["columns"] = list(tableau_distribution.columns)
tableau["show"] = "headings"

# Définir les en-têtes
for col in tableau_distribution.columns:
    tableau.heading(col, text=col)
    tableau.column(col, width=150)

# Remplir le tableau
for index, row in tableau_distribution.iterrows():
    tableau.insert("", "end", values=list(row))

tableau.pack(expand=True, fill="both")

# → Graphique
plt.figure(figsize=(8, 5))
sns.histplot(df["TotalAmount"], kde=True)
```

```
plt.title("Distribution des montants de panier")
plt.xlabel("Montant (€)")
plt.ylabel("Nombre de paniers")
plt.grid()
plt.show()

# Lancer la fenêtre
fenetre.mainloop()

# (Optionnel) Export en CSV pour Word
tableau_distribution.to_csv("tableau_distribution.csv", index=False)

curseur.close()
connexion.close()

# ÉTAPE 3 – Analyse des données
# → Calculs statistiques
moyenne = df["TotalAmount"].mean()
ecart_type = df["TotalAmount"].std()

print(f"Moyenne : {moyenne:.2f} €")
print(f"Écart-type : {ecart_type:.2f} €")
```

ANALYSE : STOCKS CRITIQUES

```
import mysql.connector
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import tkinter as tk
from tkinter import ttk

# Connexion à la base
connexion = mysql.connector.connect(
    host="localhost",
    user="root",
    password="",
    database="supermarche"
)

curseur = connexion.cursor()
requete = "SELECT * FROM stock"
curseur.execute(requete)
colonnes = [desc[0] for desc in curseur.description]
resultats = curseur.fetchall()
df_stock = pd.DataFrame(resultats, columns=colonnes)

# Nettoyage
```

```
df_stock["QuantityAvailable"] = df_stock["QuantityAvailable"].astype(int)

# ✔️ Calcul du minimum
stock_min = df_stock["QuantityAvailable"].min()
print(f"Quantité minimale disponible en stock : {stock_min} unités")

# ✔️ Produits ayant le stock minimum
produits_min = df_stock[df_stock["QuantityAvailable"] == stock_min]
print("\nProduits ayant la quantité minimale en stock :")
print(produits_min[["idStock", "QuantityAvailable", "Product_idProduct", "Location"]])


# ✔️ Seuil critique fixé à 100
seuil_critique = 100
df_critique = df_stock[df_stock["QuantityAvailable"] < seuil_critique]
print(f"\nProduits en-dessous du seuil critique ({seuil_critique} unités) :")
print(df_critique[["idStock", "QuantityAvailable", "Product_idProduct", "Location"]])

# --- FENÊTRE 1 : Histogramme
plt.figure(figsize=(8, 4))
sns.histplot(df_stock["QuantityAvailable"], bins=15, kde=False, color="salmon")
plt.axvline(stock_min, color="red", linestyle="--", label=f"Minimum : {stock_min}")
plt.axvline(seuil_critique, color="orange", linestyle="--", label=f"Seuil critique : {seuil_critique}")
plt.title("Répartition des quantités disponibles en stock")
plt.xlabel("Quantité disponible")
plt.ylabel("Nombre de produits")
plt.legend()
plt.grid()
plt.tight_layout()
plt.show()

# --- FENÊTRE 2 : Top 5 des plus faibles
fenetre = tk.Tk()
fenetre.title("Top 5 des stocks les plus faibles")

tableau = ttk.Treeview(fenetre)
tableau["columns"] = ["idStock", "QuantityAvailable", "Product_idProduct", "Location"]
tableau["show"] = "headings"

for col in tableau["columns"]:
    tableau.heading(col, text=col)
    tableau.column(col, width=150)




# --- FENÊTRE 3 : Produits en-dessous du seuil critique
fenetre_critique = tk.Toplevel()
fenetre_critique.title("Produits en-dessous du seuil critique")

tableau_critique = ttk.Treeview(fenetre_critique)
```

```
tableau_critique["columns"] = ["idStock", "QuantityAvailable", "Product_idProduct", "Location"]
tableau_critique["show"] = "headings"

for col in tableau_critique["columns"]:
    tableau_critique.heading(col, text=col)
    tableau_critique.column(col, width=150)

for _, row in df_critique.iterrows():
    tableau_critique.insert("", "end", values=list(row[["idStock", "QuantityAvailable", "Product_idProduct",
"Location"]]))

tableau_critique.pack(expand=True, fill='both')

# Lancer la fenêtre principale (qui gardera tout affiché)
fenetre.mainloop()

# Fermeture de la base
curseur.close()
connexion.close()
```

ANALYSE : TENSION ENTRE VENTES ET STOCK(DISTRIBUTION STATISTIQUE BILLINE)

```
import pandas as pd
import mysql.connector
import tkinter as tk
from tkinter import ttk

# Connexion à la base MySQL
conn = mysql.connector.connect(
    host="localhost",
    user="root",
    password="",
    database="supermarche"
)

# Récupération des données
query = "SELECT Quantity FROM billline"
df = pd.read_sql(query, conn)
conn.close()

# Calcul de la distribution statistique
distribution = df['Quantity'].value_counts().sort_index().reset_index()
distribution.columns = ['XI', 'Effectif']
distribution['Fréquence'] = distribution['Effectif'] / distribution['Effectif'].sum()
distribution['Pourcentage'] = distribution['Fréquence'] * 100

# Création de la fenêtre principale
```

```
root = tk.Tk()
root.title("Distribution statistique - Quantity")

# Création du tableau (Treeview)
tree = ttk.Treeview(root, columns=('XI', 'Effectif', 'Fréquence', 'Pourcentage'), show='headings')

# Entêtes
tree.heading('XI', text='XI')
tree.heading('Effectif', text='Effectif')
tree.heading('Fréquence', text='Fréquence')
tree.heading('Pourcentage', text='Pourcentage (%)')

# Insertion des lignes
for _, row in distribution.iterrows():
    tree.insert('', 'end', values=(
        int(row['XI']),
        int(row['Effectif']),
        f"{row['Fréquence']:.3f}",
        f"{row['Pourcentage']:.2f}"
    ))

# Mise en page
tree.pack(padx=10, pady=10, fill='both', expand=True)

# Lancement de la boucle principale
root.mainloop()
```

ANALYSE : TENSION ENTRE VENTES ET STOCK(DISTRIBUTION STATISTIQUE STOCK)

```
import pandas as pd
import mysql.connector
import tkinter as tk
from tkinter import ttk

# Connexion à la base MySQL
conn = mysql.connector.connect(
    host="localhost", user="root", password="", database="supermarche"
)

# Requête SQL pour récupérer QuantityAvailable dans la table Stock
query = "SELECT QuantityAvailable FROM Stock"
df = pd.read_sql(query, conn)
conn.close()

# Calcul de la distribution statistique
```

```
distribution = df["QuantityAvailable"].value_counts().sort_index().reset_index()
distribution.columns = ["XI", "Effectif"]
distribution["Fréquence"] = distribution["Effectif"] / distribution["Effectif"].sum()
distribution["Pourcentage"] = distribution["Fréquence"] * 100

# Interface graphique avec Tkinter
root = tk.Tk()
root.title("Distribution statistique - QuantityAvailable (Stock)")

# Tableau Treeview
tree = ttk.Treeview(
    root, columns=("XI", "Effectif", "Fréquence", "Pourcentage"), show="headings"
)

# Entêtes de colonnes
tree.heading("XI", text="XI")
tree.heading("Effectif", text="Effectif")
tree.heading("Fréquence", text="Fréquence")
tree.heading("Pourcentage", text="Pourcentage (%)")

# Remplissage du tableau
for _, row in distribution.iterrows():
    tree.insert(
        "",
        "end",
        values=(
            int(row["XI"]),
            int(row["Effectif"]),
            f"{row['Fréquence']:.3f}",
            f"{row['Pourcentage']:.2f}",
        ),
    )

# Affichage
tree.pack(padx=10, pady=10, fill="both", expand=True)
root.mainloop()
```

ANALYSE : TENSION ENTRE VENTES ET STOCK(COEFFICIENT DE PEARSON)

```
import pandas as pd
import mysql.connector
import tkinter as tk
from tkinter import ttk
from matplotlib.backends.backend_tkagg import FigureCanvasTkAgg
import matplotlib.pyplot as plt
```

```
from scipy.stats import pearsonr

# Connexion à la base MySQL
conn = mysql.connector.connect(
    host="localhost",
    user="root",
    password="",
    database="supermarche"
)

# Requête SQL : jointure entre billline et stock via product_id
query = """
SELECT b.Product_idProduct, b.Quantity, s.QuantityAvailable
FROM billline b
JOIN stock s ON b.Product_idProduct = s.Product_idProduct
"""

df = pd.read_sql(query, conn)
conn.close()

# Calcul du coefficient de corrélation de Pearson
r, p_value = pearsonr(df['Quantity'], df['QuantityAvailable'])

# Création de l'interface Tkinter
root = tk.Tk()
root.title("Corrélation Quantity vs QuantityAvailable")

# Ajout d'un label avec le coefficient
label = ttk.Label(root, text=f"Coefficient de corrélation de Pearson : {r:.4f}")
label.pack(pady=10)

# Création du nuage de points
fig, ax = plt.subplots(figsize=(6, 4))
ax.scatter(df['Quantity'], df['QuantityAvailable'], color='blue')
ax.set_xlabel('Quantity (vendue)')
ax.set_ylabel('Quantity Available (stock)')
ax.set_title("Nuage de points : Ventes vs Stock")

# Intégration du graphe dans Tkinter
canvas = FigureCanvasTkAgg(fig, master=root)
canvas.draw()
canvas.get_tk_widget().pack(padx=10, pady=10)

# Lancement de l'application
root.mainloop()
```